Aula 9 – Tabelas de Dispersão (Hash Tables)

Tabelas de Hash

Funcões de Hash

Resolução de Colisões

Tabelas de Hash

Pilhas: Inserção O(1), Consulta O(1), Eliminação O(1)

Listas Ligadas: Inserção O(1), Consulta O(N), Eliminação O(N)

Árvores: Inserção O(logn), Consulta O(logn), Eliminação O(logn)

Heaps: Inserção O(logn), Consulta O(1), Eliminação O(logn)

Filas: Inserção O(1), Consulta O(1), Eliminação O(1)

Hash: Inserção O(1), Consulta O(1), Eliminação O(1)

Tabelas de Hash

- Vantagens
- Eficiência
  - Estruturas com algoritmos de complexidade O(1).
- Desvantagens
- Não existe a noção de máximo e de mínimo.
- Não é possível fazer listagens ordenadas.
- Definição à priori do tamanho da estrutura.

- A eficiência paga-se com o desperdício de memória
- Quanto maior for o desperdício mais eficiente se torna a estrutura.

- Implementação
- Array
  - Acesso directo aos elementos

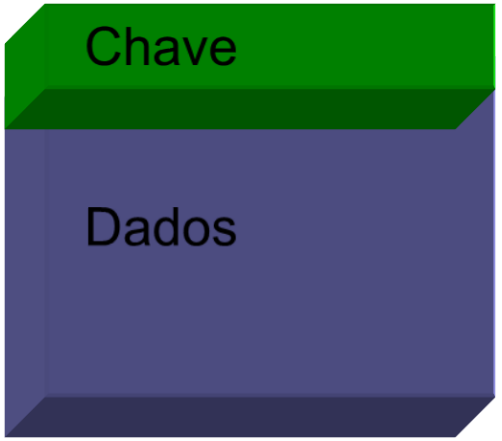
Aplicação

Construir uma base de dados com informação acerca dos alunos da nossa escola. Como esta base de dados vai servir todos os serviço, da instituição , (departamentos, secretaria, etc.) tem de estar num suporte onde a manipulação seja rápida de modo a servir com eficiência todas as aplicações que dela dependem e são muitas.

Ênfase na eficiência

Resolução

- Array de 20.000 elementos
- Chave númérica
  - Número do aluno
- dados
  - Nome, morada, etc.
- Introdução
- Introduzir o elemento na posição da chave
- Eliminar
- Colocar a chave null
- Pesquisar
- Retornar o elemento que está na posição “chave



Implementação - Aluno

```
public class Aluno {
    int numero;
    String nome;

    Aluno(int id,String n) {
        numero=id;
        nome=n;
    }
     public String toString() {
        return numero + " - " + nome;
    }

   public int hashCode() {
        return numero;
    }
}
```

Implementação - Tabela

```
class Hash {
    Object array[];
    Hash(int n) {
        array = new Object[n];
    }
     void insert(Object o) {
        array[o.hashCode()]=o;
    }
    void delete(Object o) {
        array[o.hashCode()]=null;
    }
    boolean search(Object o) {
        return array[o.hashCode()]!=null;
    }
    Object get(int num) {
        return array[num];
    }
 }
```

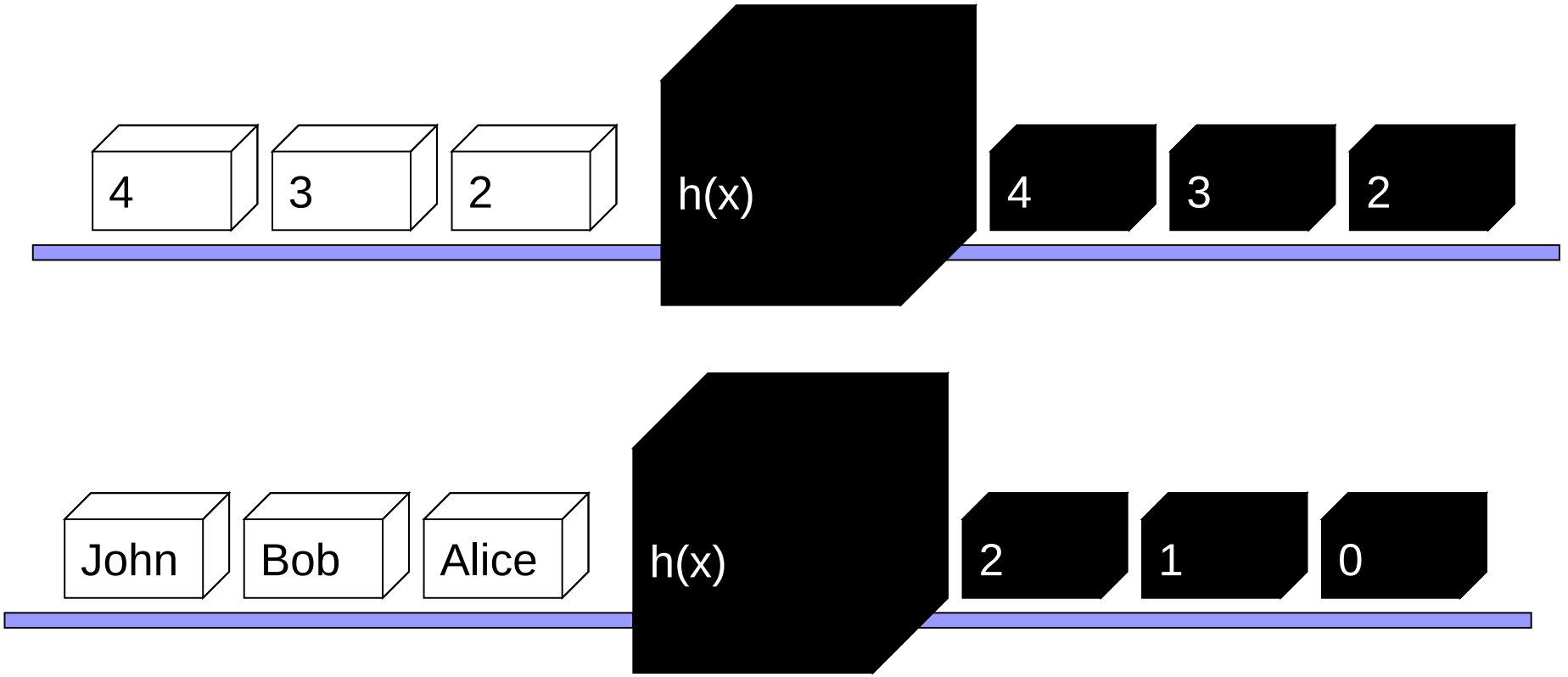
Teste

```
Hash ipt = new Hash(16);
ipt.insert(new Aluno(1,"Jose"));
ipt.insert(new Aluno(3,"Maria"));
ipt.insert(new Aluno(7,"Antonio"));
ipt.insert(new Aluno(10,"Manuel"));
ipt.insert(new Aluno(12,"Tiago"));
ipt.insert(new Aluno(14,"Daniel"));
ipt.insert(new Aluno(15,"Gonçalo"));
ipt.insert(new Aluno(5,"Inês"));
```

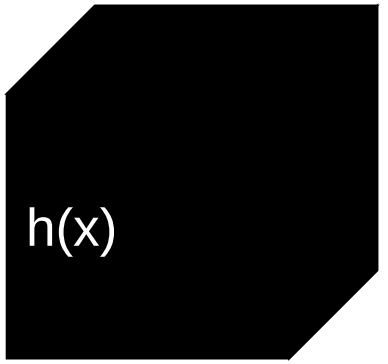
```
[0]
[1]     1 - Jose
[2]
[3]
[4]
[5]     5 - Inês
[6]
[7]     7 - Antonio
[8]
[9]
[10] 10 - Manuel
[11]
[12] 12 - Tiago
[13]
[14] 14 - Daniel
[15] 15 - Gonçalo
```

Funções de Hash

- **`public int hashCode()`**
- ***Método que permite converter um objecto num numero***



Exemplos de Funções de Hash



- Números inteiros

$$1527 = 7 + 2*10 + 5 * 100 + 1 * 1000$$

$$= 7 * 10^0 + 2 * 10^1 + 5 * 10^2 + 1 * 10^3$$

$$taco = 19 * 25^0 + 1 * 25^1 + 3 * 25^2 + 14 * 25^3$$

$$= 19 + 25 + 3 * 625 + 14 * 14 * 15625$$

- Palavras

Taco= 220 669

...	tacm	tacn	taco	tacp	tacq	tacr	...
	220667	220668	220669	220670	220671	220672	

Boas funções de Hash

- Devem ser **deterministicas**
- Para o mesmo input devem fornecer o mesmo output
- Devem promover a dispersão dos elementos na tabela
- Evitar colisões entre elementos distintos
- Devem ser facilmente computáveis
- Promover a eficiência

Resolução Colisões - Método Linear

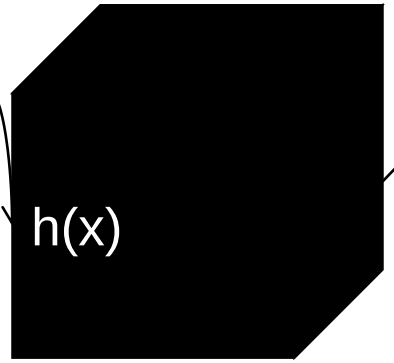
Quando uma célula está ocupada procura-se a primeira que esteja livre

Usando a função de hash:

numero de aluno % 10

Introdução de elementos

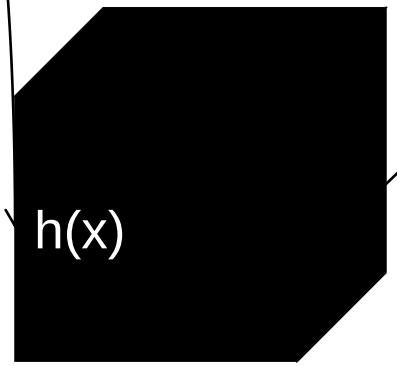
```
HashLinear ipt = new HashLinear(10);
ipt.insert(new Aluno(1,"Jose"));
ipt.insert(new Aluno(3,"Maria"));
ipt.insert(new Aluno(7,"Antonio"));
ipt.insert(new Aluno(10,"Manuel"));
ipt.insert(new Aluno(12,"Tiago"));
ipt.insert(new Aluno(14,"Daniel"));
ipt.insert(new Aluno(15,"Gonçalo"));
```



0	10 -manuel
1	1-jose
2	12 - tiago
3	3-maria
4	14 - daniel
5	15 - gonçalo
6	
7	7- antonio
8	
9	

Introdução de elementos

```
ipt.insert(new Aluno(5,"Inês"));
```

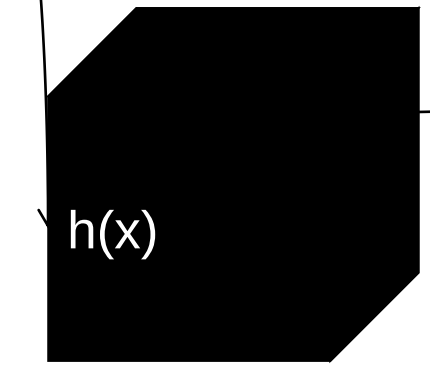


5 - 6

0	0 - Antonio
1	1-jose
2	12 - tiago
3	3-maria
4	14 - daniel
5	15 - gonçalo
6	**5 - Inês**
7	7- Antonio
8	
9	

Introdução de elementos

```
ipt.insert(new Aluno(55,“Sara"));
```



5 – 6 - 8

0	0 - Antonio
1	1-jose
2	12 - tiago
3	3-maria
4	14 - daniel
5	15 - gonçalo
6	**5 - Inês**
7	7- Antonio
8	**55 - Sara**
9	

Vantagens de desvantagens

- Vantagens
- Eficiencia de espaço
  - Os valores são guardados na própria tabela.
- Simples
  - Não requer nunhuma estrutura auxilar
- Funciona bem quando quando o factor de carga é baixo

- Desvantagens:
- Como os elementos em colisão ocupam posições adjacentes formam-se blocos que prejudicam a eficiencia
- A performançe degrada-se com um factor de carga que exceda os 75%
- Baixa eficiencia com a formação de clusters
- Desperdiça muito espaço

Resolução de Colisões - Método Quadrático

- Os duplicados não são introduzidos com a mesma função de hash
- Objectivo : por os elementos duplicados o mais dispersos possível

- Assumindo que a função de hash é

numero de aluno %10

- Se existir uma colisão usa-se um função quadrática
- Sendo x o resultado da função de hash, as posições seguintes são respectivamente $x+1^2$, $x+2^2$, $x+3^2$, $x+4^2$, $x+5^2$, etc.

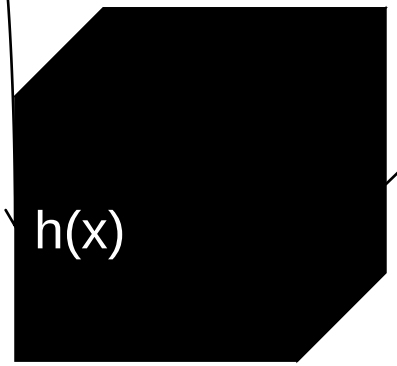
Introdução de elementos

$x+1^2$, $x+2^2$, $x+3^2$, $x+4^2$, $x+5^2$

```
ipt.insert(new Aluno(5,"Inês"));
```

X%10 = 5

(5+1^2) % 10 = 6



0	0 - Antonio
1	1-jose
2	12 - tiago
3	3-maria
4	14 - daniel
5	15 - gonçalo
6	**5 - Inês**
7	7- Antonio
8	
9	

Introdução de elementos

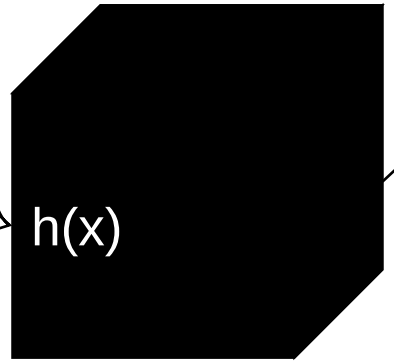
$x+1^2$, $x+2^2$, $x+3^2$, $x+4^2$, $x+5^2$

```
ipt.insert(new Aluno(55,“Sara"));
```

X = 55 %10 = 5

$(5+1^2)$ % 10 = 6

$(5+2^2)$ % 10 = 9



0	0 - Antonio
1	1-jose
2	12 - tiago
3	3-maria
4	14 - daniel
5	15 - gonçalo
6	**5 - Inês**
7	7- Antonio
8	
9	**55 - Sara**

Resolução de Colisões - Método Duplo

- Os duplicados não são introduzidos com a mesma função de hash
- Objectivo : por os elementos duplicados o mais dispersos possível

- Assumindo que a função de hash é

numero de aluno %10

- Se existir uma colisão usa-se a seguinte função
- Sendo x o resultado da função de hash e y o maior número primo inferior ao número máximo de elementos, as posições seguintes são respectivamente (x+1*z)%10, (x+2*z)%10, (x+3*z)%10, (x+4*z)%10, … em que z = y – (numero de aluno % y)

Introdução de elementos

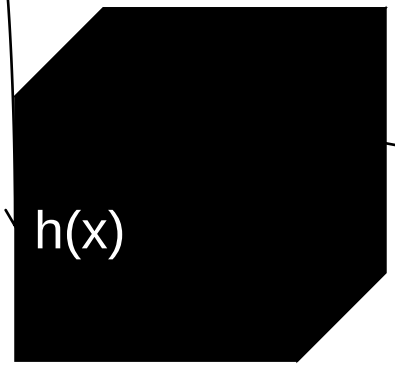
(x+1*z)%10, (x+2*z)%10, (x+z*y)%10, (x+z*y)%10, ...

z = y - (numero de aluno % y)

```
ipt.insert(new Aluno(5,"Inês"));
```

x = 5 % 10 = 5
y = 7
z = 7 - (5 % 7) = 2

(5+1*2)%10 = 7
(5+2x2)%10 = 9



0	0 - Antonio
1	1-jose
2	12 - tiago
3	3-maria
4	14 - daniel
5	15 - gonçalo
6	
7	7- Antonio
8	
9	5 - Inês

Introdução de elementos

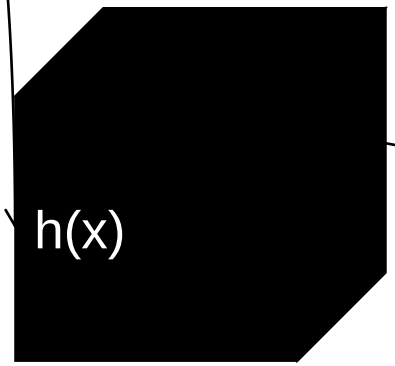
(x+1*z)%10, (x+2*z)%10, (x+z*y)%10, (x+z*y)%10, ...

z = y - (numero de aluno % y)

```
ipt.insert(new Aluno(55,"Sara"));
```

x = 55 % 10 = 5
y = 7
z = 7 - (55 % 7) = 1

(5+1*1)%10 = 6



0	0 - Antonio
1	1 - jose
2	12 - tiago
3	3 - maria
4	14 - daniel
5	15 - gonçalo
6	55 - Sara
7	7- Antonio
8	
9	5 - Inês

Hashing Duplo - Exercicio

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

- Inserir, usando a função de hash x%10 e resolvendo as colisões usando o método duplo:
  - 89
  - 18
  - 49
  - 58
  - 69
  - 60

- Tentar inserir o 23

Hashing Duplo - Exercicio

0	69
1	
2	
3	58
4	
5	
6	49
7	
8	18
9	89

- 89 % 10 = 9
- 18 % 10 = 8
- 49 % 10 = 9 ← Colisão
- 7 - (49 % 7) = 7 – 0 = 7
- (9 + 1 * 7) % 10 = 6
- 58 % 10 = 8 ← Colisão
- 7 - (58 % 7) = 7 – 2 = 5
- (8 + 1 * 5) % 10 = 3
- 69 % 10 = 9 ← Colisão
- 7 - (69 % 7) = 7 – 6 = 1
- (9 + 1 * 1) % 10 = 0

Hashing Duplo - Exercicio

0	69
1	
2	60
3	58
4	
5	
6	49
7	
8	18
9	89

- 60 % 10 = 0 ← Colisão
- 7 - (60 % 7) = 7 – 4 = 3
- (0 + 1 * 3) % 10 = 3 ← Colisão
- (0 + 2 * 3) % 10 = 6 ← Colisão
- (0 + 3 * 3) % 10 = 9 ← Colisão
- (0 + 4 * 3) % 10 = 2
- 23 % 10 = 3 ← Colisão
- 7 - (23 % 7) = 7 – 2 = 5
- (3 + 1 * 5) % 10 = 8 ← Colisão
- (3 + 2 * 5) % 10 = 3 ← Colisão
- (3 + 3 * 5) % 10 = 8 ← Colisão
- (3 + 4 * 5) % 10 = 3 ← Colisão
- (3 + 5 * 5) % 10 = 8 ← Colisão
- (3 + 6 * 5) % 10 = 3 ← Colisão e agora??

Rehashing

- Por vezes torna-se necessário aumentar o tamanho da tabela de hash:
- Quando começa a ocorrer muitas colisões (caso do numero 60)
- Quando entramos em ciclo infinito e não conseguimos colocar o valor (caso do numero 23)

- Quando começa a ocorrer muita colisões, deve-se ao facto do factor de carga do hash ser muito elevado, assim:
- Se o factor de carga ultrapassar um determinado valor (por exemplo 0,5), "duplica-se" o tamanho da tabela
- A Inserção ocasionalmente sera O(N), porque terá que voltar a inserir todos os valores

Rehashing

0	69
1	
2	60
3	58
4	
5	
6	49
7	
8	18
9	89

- Inserir:
- 23

Factor de carga = 6/10 > 0,5

Novo tamanho da Tabela 20

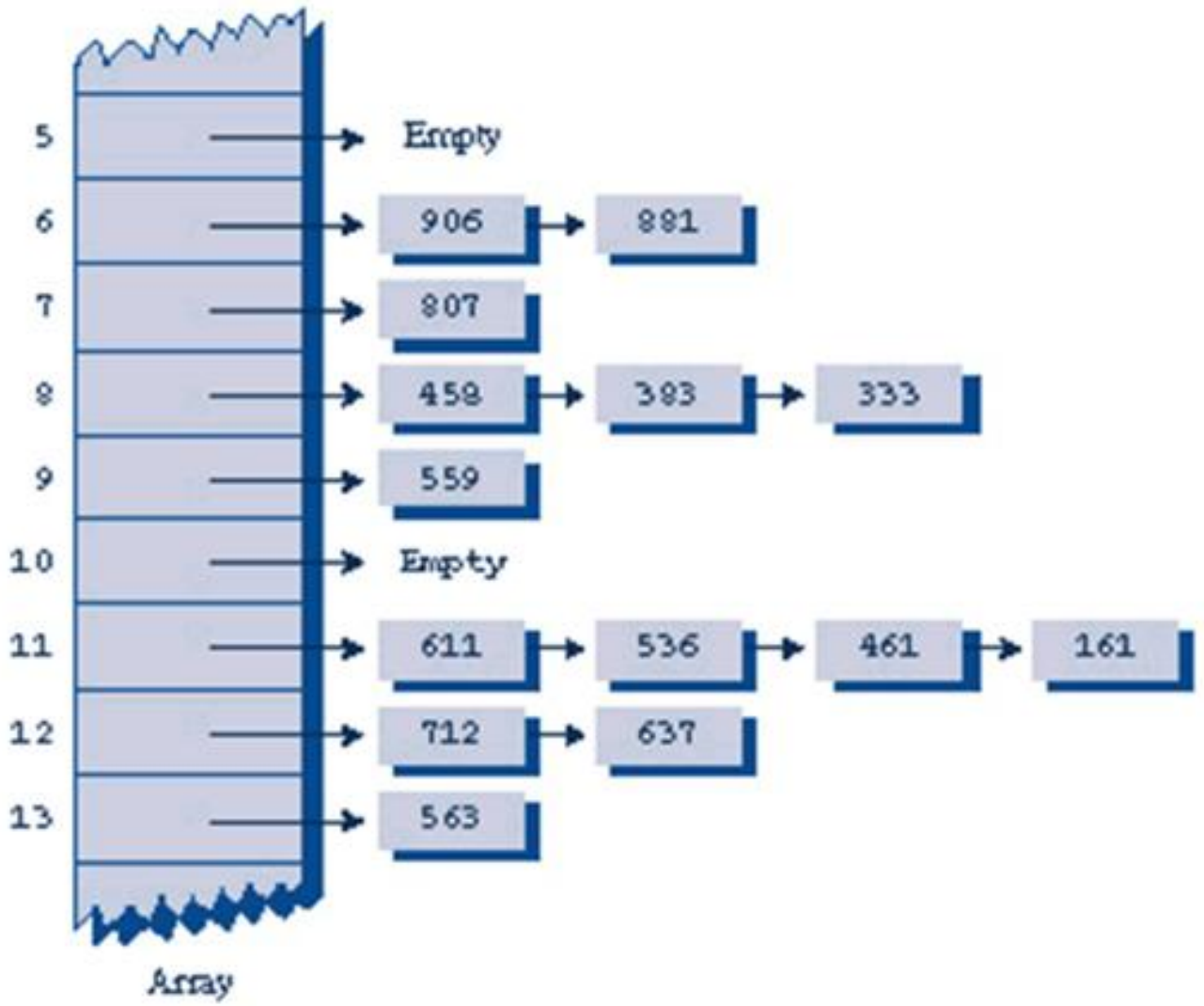
y = 19, maior primo menor que o tamanho máximo

z = 19 - (X % 19)

Volta a inserir todos os elementos

Outras Abordagens às colisões

- Array de contentores
- Os duplicados ficam nos contentores
- Contentores
- Vectores
- Listas
- Arvores



Teste

```
[0] =[10 - Manuel]
[1] =[1 - Jose]
[2] =[12 - Tiago]
[3] =[3 - Maria]
[4] =[14 - Daniel]
[5] =[15 - Gonçalo, 5 - Inês, 55 - Sara]
[6] =[]
[7] =[7 - Antonio]
[8] =[]
[9] =[]
```